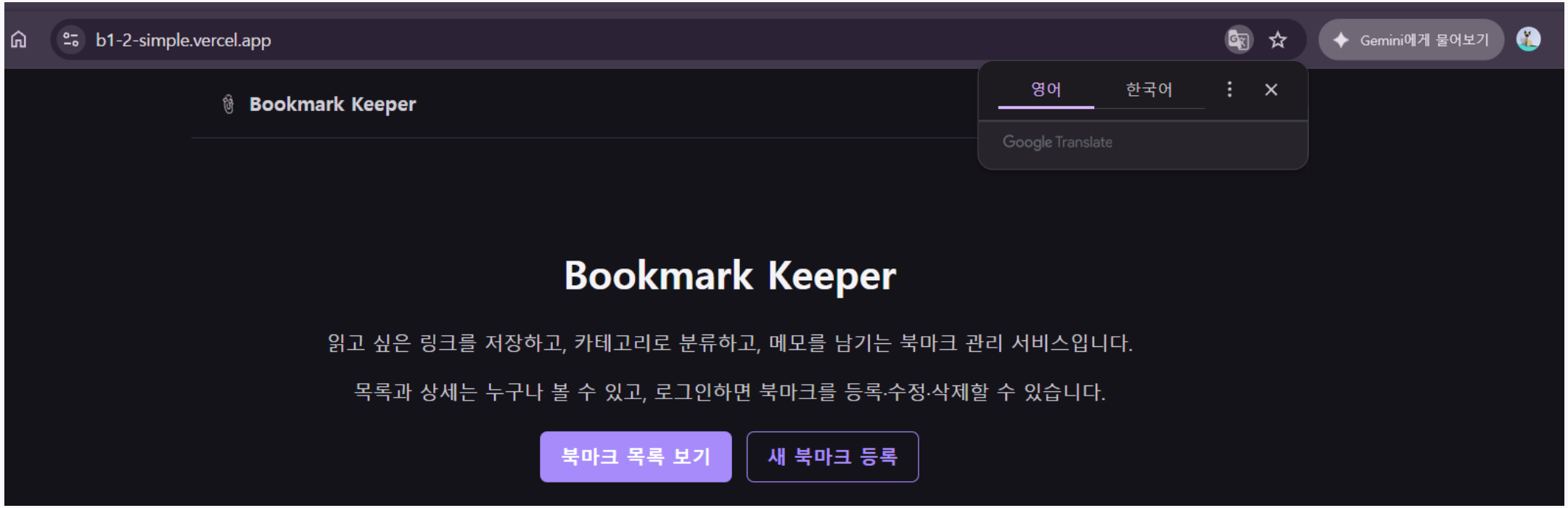
# B1-2 평가 발표

Bookmark Keeper

평가 1-1부터 5-1까지 17개 문항



평가 1-1

# 라우트와 Not Found

| 경로 | 화면 | 접근 |
| --- | --- | --- |
| / | 홈 | 공개 |
| /bookmarks | 목록 | 공개 |
| /bookmarks/:id | 상세 | 공개 |
| /bookmarks/new | 등록 | 로그인 |
| /bookmarks/:id/edit | 수정 | 로그인 |
| /login | 로그인·회원가입 | 공개 |
| /profile | 내 프로필 | 로그인 |
| * | Not Found | 공개 |

근거: src/App.jsx, src/components/ProtectedRoute.jsx, vercel.json

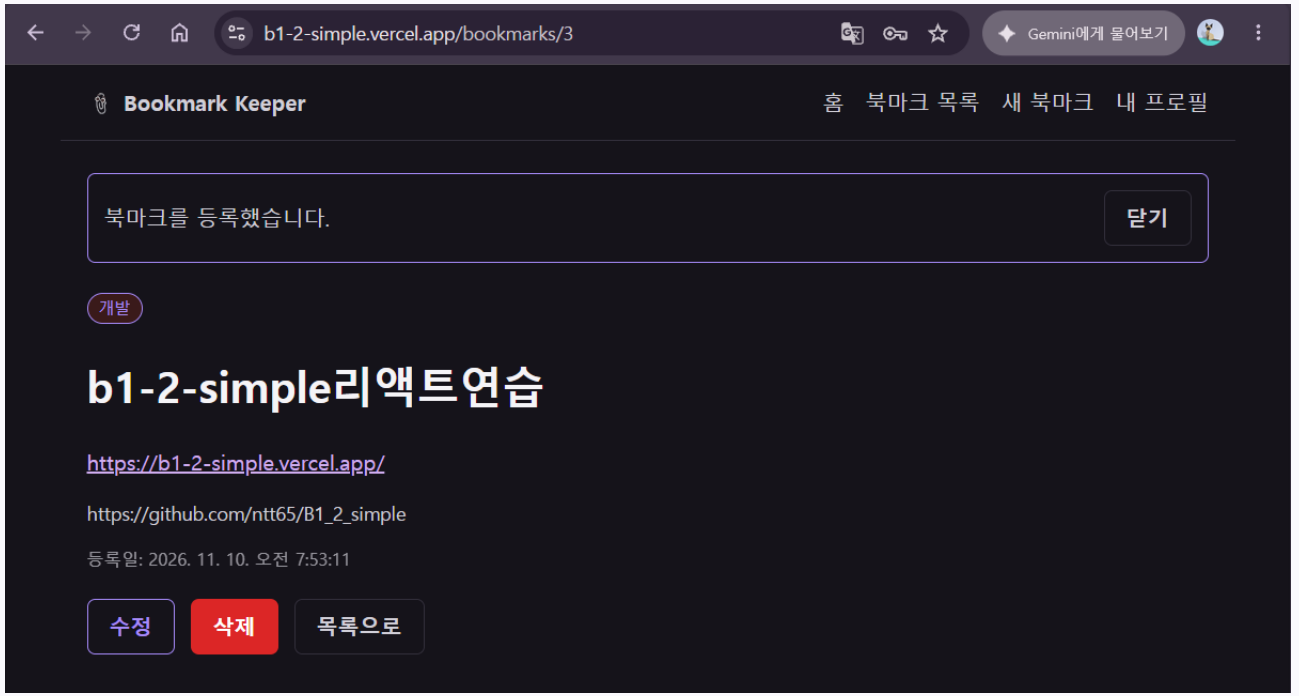
평가 1-2

# CRUD의 성공 조건

Supabase bookmarks에서 원격 CRUD를 실행합니다.

등록은 새 상세로, 삭제는 목록으로 이동합니다.

수정·삭제의 반환 0행을 실패로 처리합니다.




근거: src/hooks/useBookmarks.js, src/pages/BookmarkDetailPage.jsx

평가 1-3

# 일관된 상태 UI

| 요청 상태 | 공통 UI와 다음 행동 |
| --- | --- |
| 확인 중 | Loading과 진행 메시지 |
| 실패 | ErrorState와 재시도 |
| 빈 목록 | EmptyState와 첫 등록 |
| 필터 결과 없음 | EmptyState와 필터 초기화 |
| 데이터 있음 | BookmarkList 또는 상세 |

근거: src/components/Loading.jsx, src/components/ErrorState.jsx, src/components/EmptyState.jsx

평가 1-4

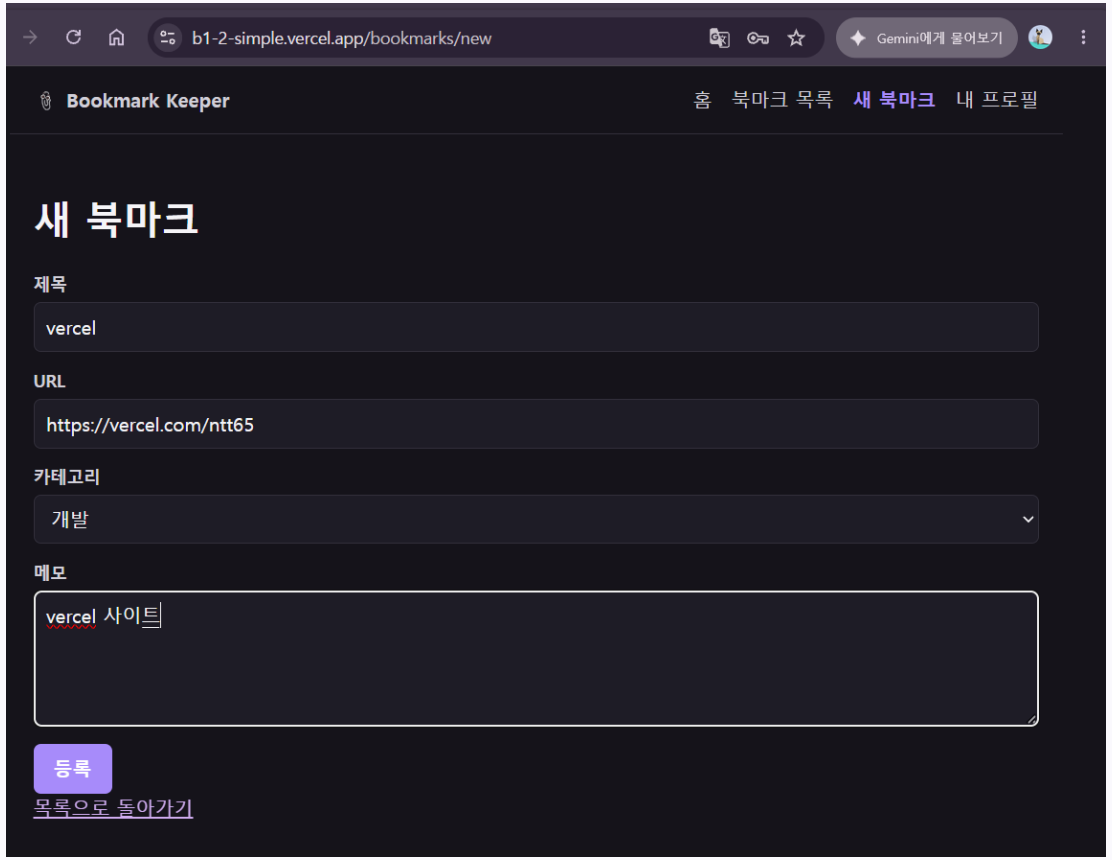
# 입력 검증과 제출 상태

필수값과 HTTP·HTTPS URL을 검사합니다.

필드 오류를 입력 근처에 표시합니다.

제출 중 버튼을 비활성화합니다.

실패한 요청에서도 입력값을 유지합니다.




근거: src/components/BookmarkForm.jsx, src/lib/bookmarkErrors.js

평가 1-5

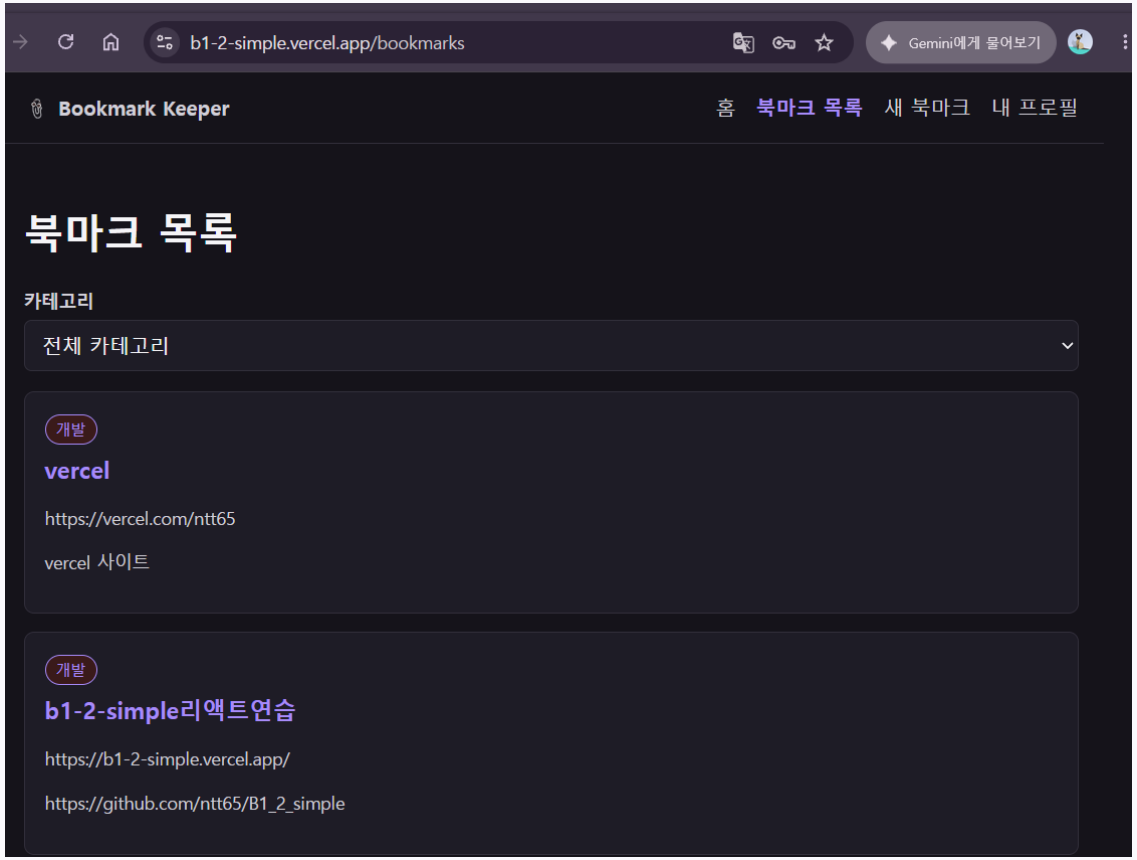
# 배포 기능의 확인 범위

현재 주소는 b1-2-simple.vercel.app입니다.

원격 API 로그인과 CRUD를 실제 반환 행까지 확인했습니다.

사용자 제공 스크린샷 7장을 README에 연결했습니다.

전체 브라우저 시나리오와 새 확인 메일은 추가 확인 대상입니다.




근거: README.md, vercel.json

# 조회 흐름의 훅 분리

useBookmarks는 목록과 조회 상태를 관리합니다.

useBookmark(id)는 상세와 수정 화면에서 재사용합니다.

데이터 요청과 상태 처리를 화면에서 분리했습니다.

```
const { bookmark, loading, error, refetch }
  = useBookmark(id);

if (loading) return <Loading />;
if (error) return <ErrorState
  onRetry={refetch} />;
```

근거: src/hooks/useBookmark.js, src/hooks/useBookmarks.js

평가 2-2

# 폴더를 나눈 이유

| 위치 | 책임 |
| --- | --- |
| pages | 화면 요청과 이동 조정 |
| components | 반복 UI와 보호 화면 |
| hooks | 조회 상태와 Context 사용 |
| context | 공유 인증 상태 |
| lib | 외부 연결·입력·복귀 경로 검증 |

근거: src/, src/lib/supabase.js

평가 2-3

# 재사용 컴포넌트의 기준

props로 동작·표시가 달라지는 13개가 있습니다.
components의 파일 수 15개와 구분합니다.
반복되는 UI와 한 가지 책임을 기준으로 나눴습니다.

| 예시 | 나눈 기준 |
| --- | --- |
| BookmarkForm | 등록·수정의 중복 제거 |
| Input / Select | label·입력·오류를 한 단위로 |
| Loading / ErrorState | 공통 상태 표시 |
| Button | variant와 loading 처리 |

근거: src/components/, src/components/BookmarkForm.jsx

평가 2-4

# 공통 상태 UI의 코드

조회 화면에서 같은 상태 UI를 사용합니다.
빈 데이터는 EmptyState로 안내합니다.
페이지는 상황별 액션을 전달합니다.

```
if (loading) return <Loading />;
if (error) return <ErrorState
  message={error}
  onRetry={refetch} />;

<BookmarkList
  bookmarks={filtered}
  emptyAction={action} />
```

근거: src/pages/BookmarksPage.jsx, src/components/BookmarkList.jsx

평가 2-5

# 훅 재사용과 인증 상태

조회 훅은 데이터와 refetch를 반환합니다.
각 훅 호출의 로컬 상태는 서로 독립적입니다.
useAuth는 AuthProvider가 가진 같은 상태를 읽습니다.
전역 상태와 훅 코드의 재사용을 구분했습니다.

근거: src/hooks/useBookmarks.js, src/hooks/useAuth.js, src/context/AuthContext.jsx

평가 3-1

# props와 state의 배치

props는 부모가 전달하는 입력, state는 컴포넌트가 관리하는 값입니다.

| 상태를 두는 곳 | 관리하는 값 |
| --- | --- |
| 조회 훅 | 목록·단건 데이터, loading, error |
| BookmarksPage | category와 파생 목록 filtered |
| BookmarkForm | values, errors, submitting |
| 페이지 | submitError, deleteError |
| AuthProvider | user, loading, sessionError |

근거: src/pages/BookmarksPage.jsx, src/components/BookmarkForm.jsx, src/context/AuthContext.jsx

# Effect와 의존성

Effect는 커밋 후 의존성 변경에 반응합니다.

단건 조회는 id가 바뀌면 다시 실행합니다.

useCallback은 함수 참조를 유지합니다.

개발 StrictMode에서는 추가 실행이 있습니다.

```
const refetch = useCallback(async () => {
  // id로 조회하고 state 갱신
}, [id]);

useEffect(() => {
  refetch();
}, [refetch]);
```

근거: src/hooks/useBookmark.js, src/main.jsx

# 비동기 요청과 화면 분기

요청 시작 시 loading을 설정합니다.
성공하면 데이터를, 실패하면 error를 저장합니다.
finally에서 loading을 해제합니다.
목록의 빈 결과는 EmptyState로, 데이터가 있으면 내용을 표시합니다.
요청 순서를 확인해 오래된 응답을 무시합니다.

근거: src/hooks/useBookmarks.js, src/hooks/useBookmark.js, tests/BookmarkQueries.test.jsx

평가 3-4

# 상태가 바꾸는 화면

| 사용자 동작·상태 | 화면의 변화 |
| --- | --- |
| 카테고리 선택 | 필터에 맞는 카드 표시 |
| 빈 제목으로 제출 | 입력 근처에 검증 오류 표시 |
| 저장 요청 중 | 버튼 비활성화와 처리 중 표시 |
| 등록 성공 | 상세 화면과 성공 알림 표시 |
| 로그인 상태 변경 | 헤더와 보호 화면 변경 |

근거: src/pages/BookmarksPage.jsx, src/components/BookmarkForm.jsx, src/components/Layout.jsx

평가 4-1

# 북마크 등록 한 기능의 흐름

| 단계 | 해당 코드와 변화 |
| --- | --- |
| 1. 라우팅 | /bookmarks/new와 ProtectedRoute |
| 2. 컴포넌트 | BookmarkNewPage와 BookmarkForm |
| 3. 입력 상태 | onChange로 values 갱신 |
| 4. 제출 이벤트 | 검증 후 세션 확인과 INSERT |
| 5. 렌더링 | 처리 중 표시, 성공 상세 또는 실패 안내 |

근거: src/App.jsx, src/components/BookmarkForm.jsx, src/pages/BookmarkNewPage.jsx

# Supabase 선택과 문제 해결

SQL 테이블과 Auth를 한 프로젝트에서 관리합니다.

memo 누락은 실제 컬럼과 payload를 맞춰 해결했습니다.

수정·삭제는 정책과 반환 행을 함께 확인했습니다.

두 앱은 같은 프로젝트의 items와 bookmarks를 따로 씁니다.

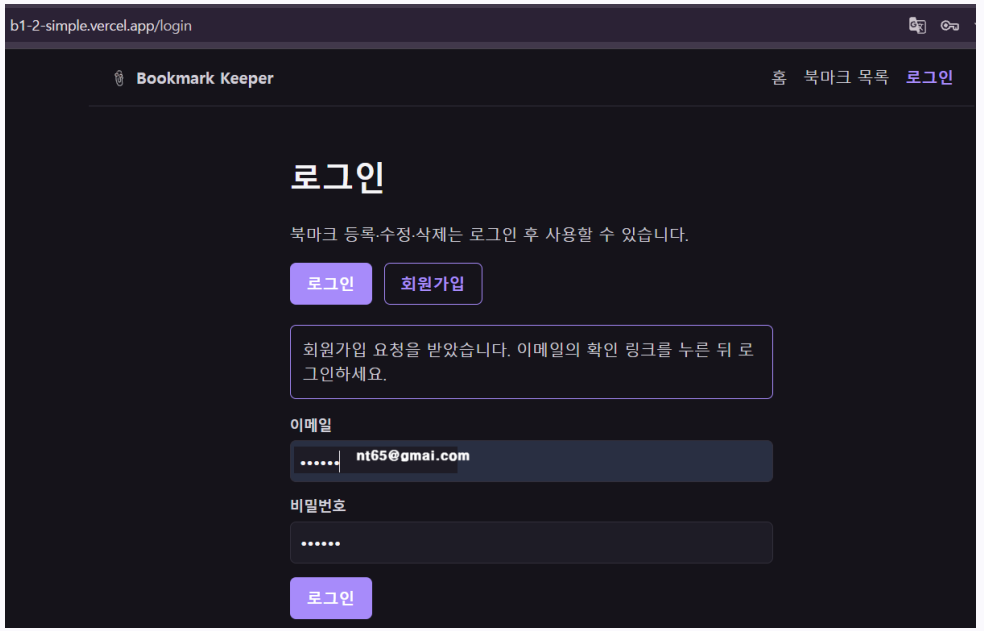
```
insert({
  title, url, memo, category,
  user_id: user.id
});

// schema.sql
// 누락 컬럼·기본값·GRANT·RLS 정렬
```

근거: src/pages/BookmarkNewPage.jsx, schema.sql, README.md

# 보너스 크레딧의 구현 근거

| 보너스 | 구현 근거 |
| --- | --- |
| 전역 상태 | AuthContext, useAuth |
| 성능 관련 | 필터 useMemo와 조회 useCallback |
| 인증 | 로그인·가입·프로필·보호 라우트 |




근거: src/context/AuthContext.jsx, src/pages/LoginPage.jsx, src/pages/BookmarksPage.jsx

평가 6

# 평가결과와 추가 확인

평가 점수와 크레딧은 평가자가 결정합니다.
자동 테스트 62개, 빌드, 린트 검사를 통과했습니다.
API 검증과 전체 브라우저 검증을 구분해 보고합니다.
신규 가입 메일의 수신·복귀는 추가 확인 대상입니다.

근거: docs/b1_2_evaluation_questions_QA.md, README.md